CS & IT ENGINEERING

COMPILER DESIGN

PARSING

Lecture No.- 01

By- Venkat sir

Recap of Previous Lecture

Topic ?? Lexical Analysis

① Tokens

② Lexical errors

Topics to be Covered

Topic ?? Parsing

Topic ?? Top down Parsing

Topic ?? Bottom up Parsing

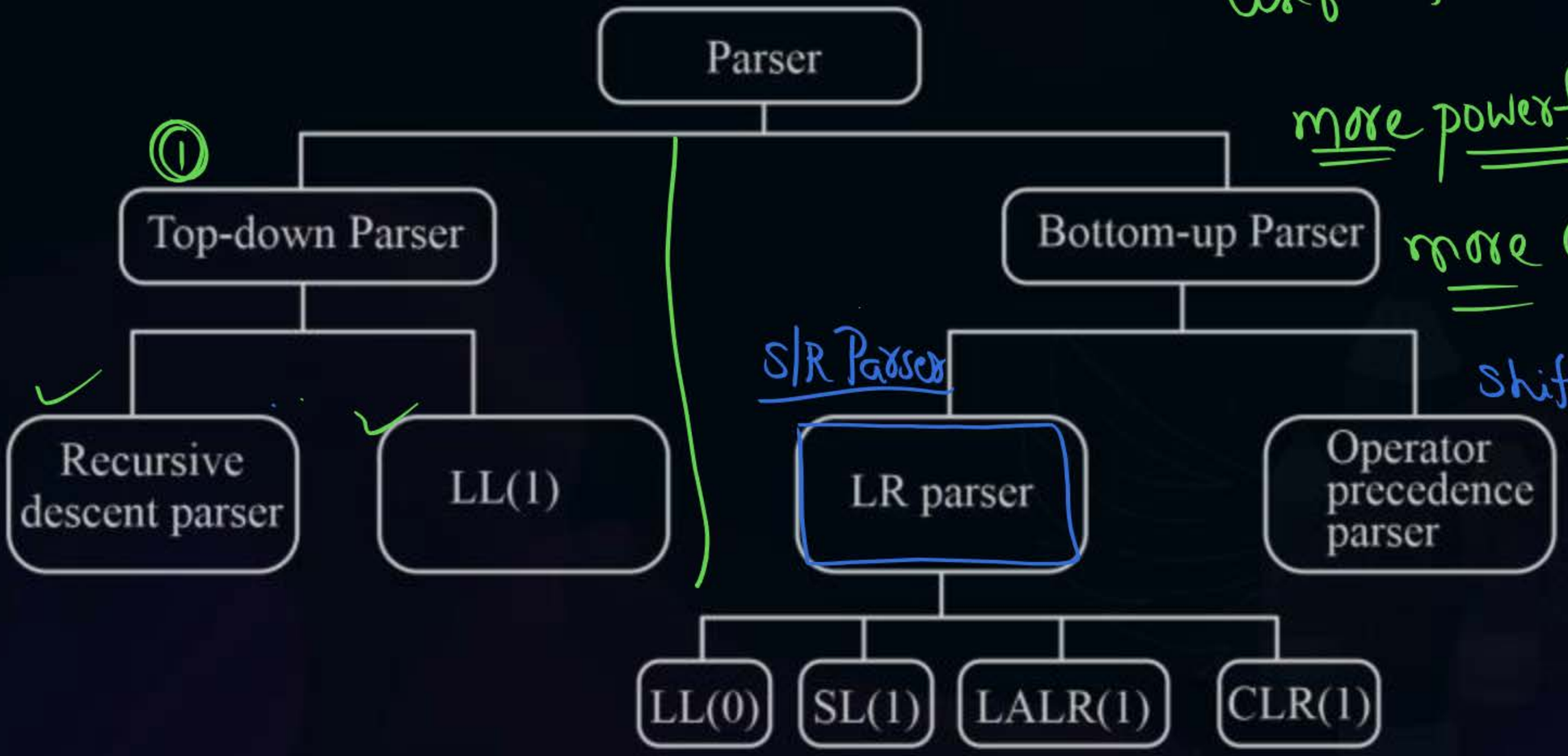
① CFG

② Parsing Algorithm

Topic : Parsing ALGORITHMS

- Parser (or) syntax analyzer is a deterministic pushdown automata.
- To design parser two types of algorithms exist known as
  1. Bottom up parsing algorithm.
  2. Top down parsing algorithm.

Topic : Parsing Algorithms



useful for Practical Compiler

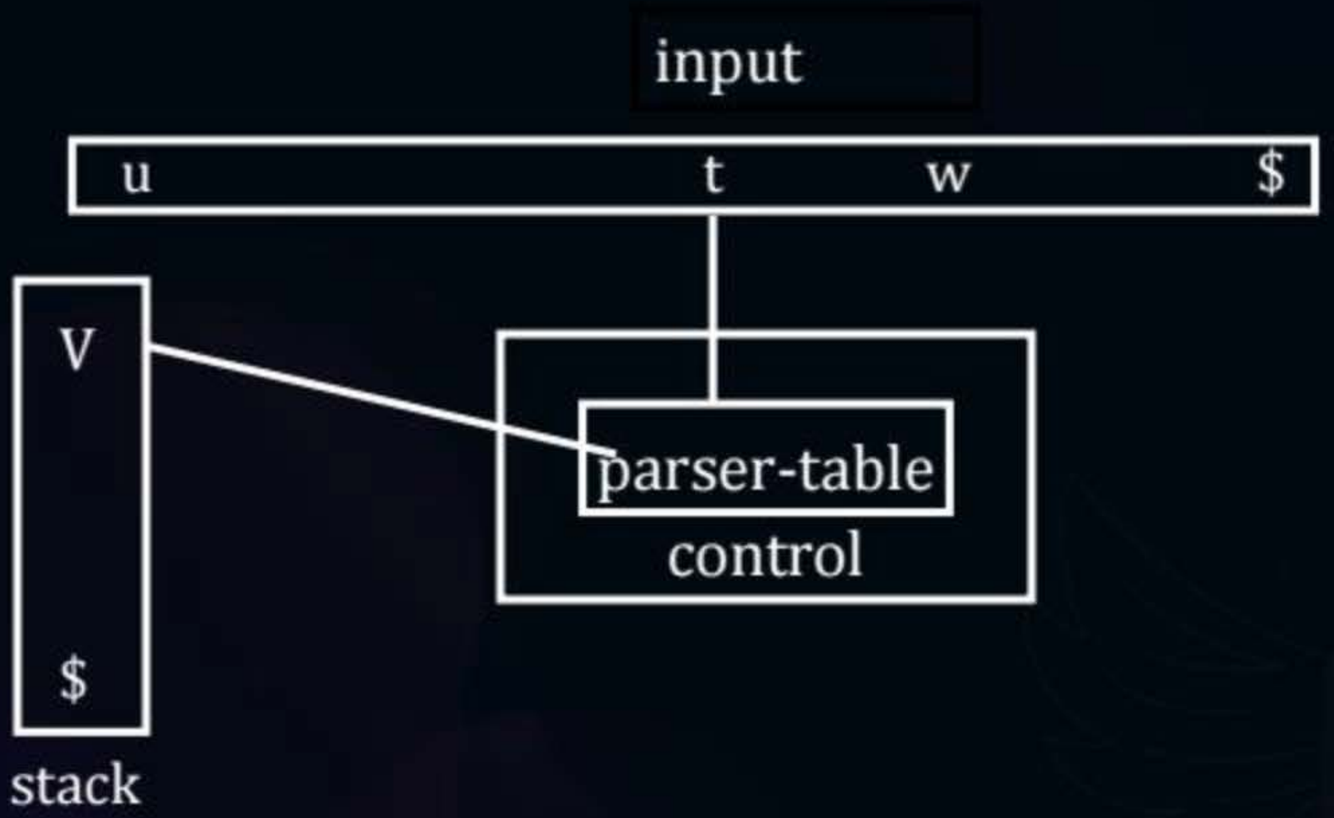
more powerful

more Complex

S/R Parser

Shift/Reduce Parser

Topic : Pushdown Automaton



Topic : Difference b/w Top-down Parsing and Bottom-up Parsing

Top–Down Parsing	Bottom–Up Parsing
A parse tree is created from root to leaves	A parse tree is created from leaves to root
Tracing leftmost derivation	Tracing rightmost derivation
Two types: Backtracking parser Predictive parser	More powerful than top-down parsing

Top down Parser

$E \rightarrow T\,E'$

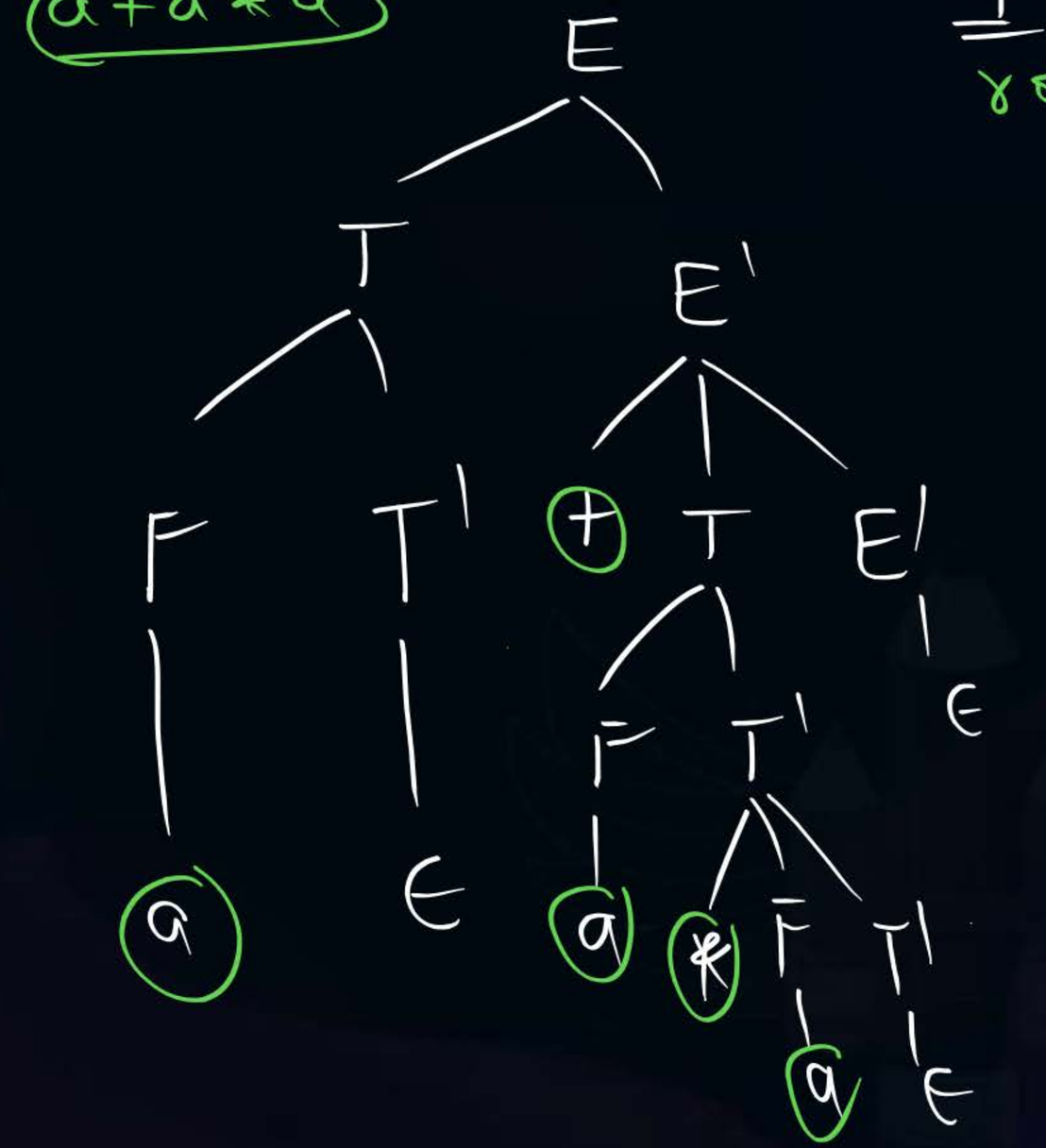
$E' \rightarrow +\,T\,E' \;(1) \mid \epsilon \;(2)$

$T \rightarrow F\,T'$

$T' \rightarrow *\,F\,T' \;(1) \mid \epsilon \;(2)$

$F \rightarrow a$

a+a*a



Topic : Bottom-Up Parsing

1. **Bottom-Up Parser :** Constructs a parse tree for an input string beginning at the leaves (the bottom) and working up towards the root (the top)
2. Bottom up parser using right most derivation in revers order
3. We can think of this process as one of "reducing" a string w to the start symbol of a grammar
4. bottom up parser performs parsing by using handle detection

Topic : Handle

- Informally, a "handle" of a string is a substring that matches the right side of the production, and whose reduction to nonterminal on the left side of the production represents one step along the reverse of a rightmost derivation
- But not every substring matches the right side of a production rule is handle.
- Formally , a "handle" of a right sentential form γ ($\equiv \alpha\beta\omega$) is a production rule $A \rightarrow \beta$ and a position of γ where the string β may be found and replaced by A to produce the previous right-sentential form in a rightmost derivation of γ.

$$S \Rightarrow \alpha A\omega \Rightarrow \alpha\beta\omega$$

- then $A \rightarrow \beta$ in the position following α is a handle of $\alpha\beta\omega$
- The string ω to the right of the handle contains only terminal symbols.

Handle 4 Handles

#Q. How many number of handles detected by bottom up parser for the following grammar and string.

S → aABe
A → Abc | b
B → d

BUP

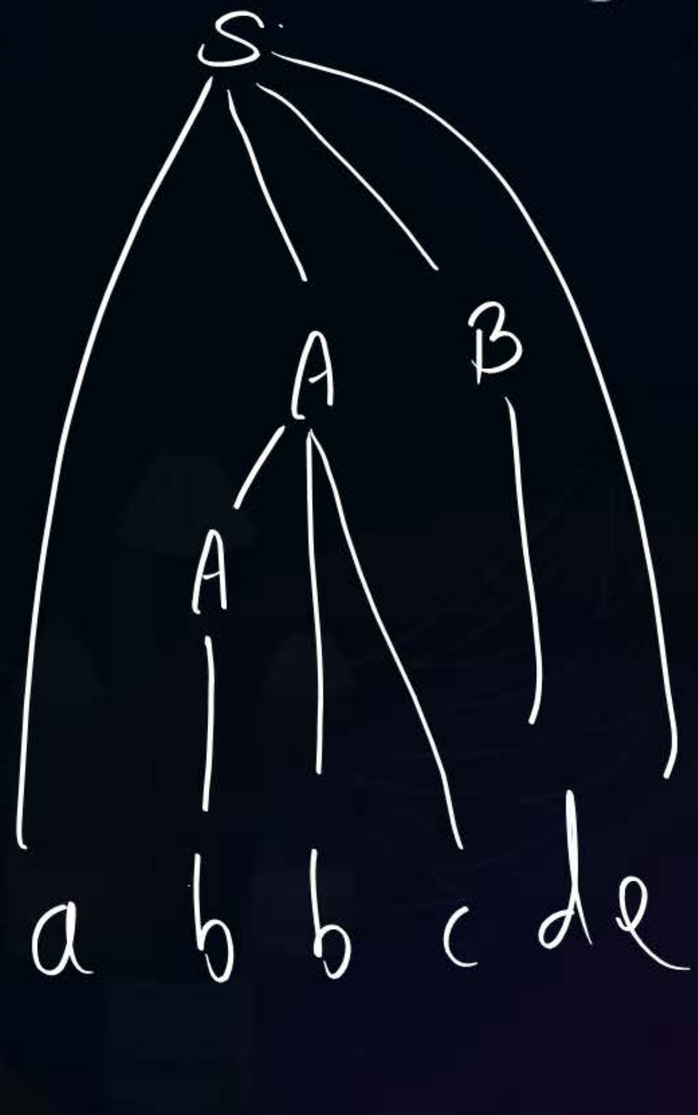
abbcde

aABe (4) a A B e

(1) (3) a A de

Abc (2) a A b c de

b (0) a b b c de



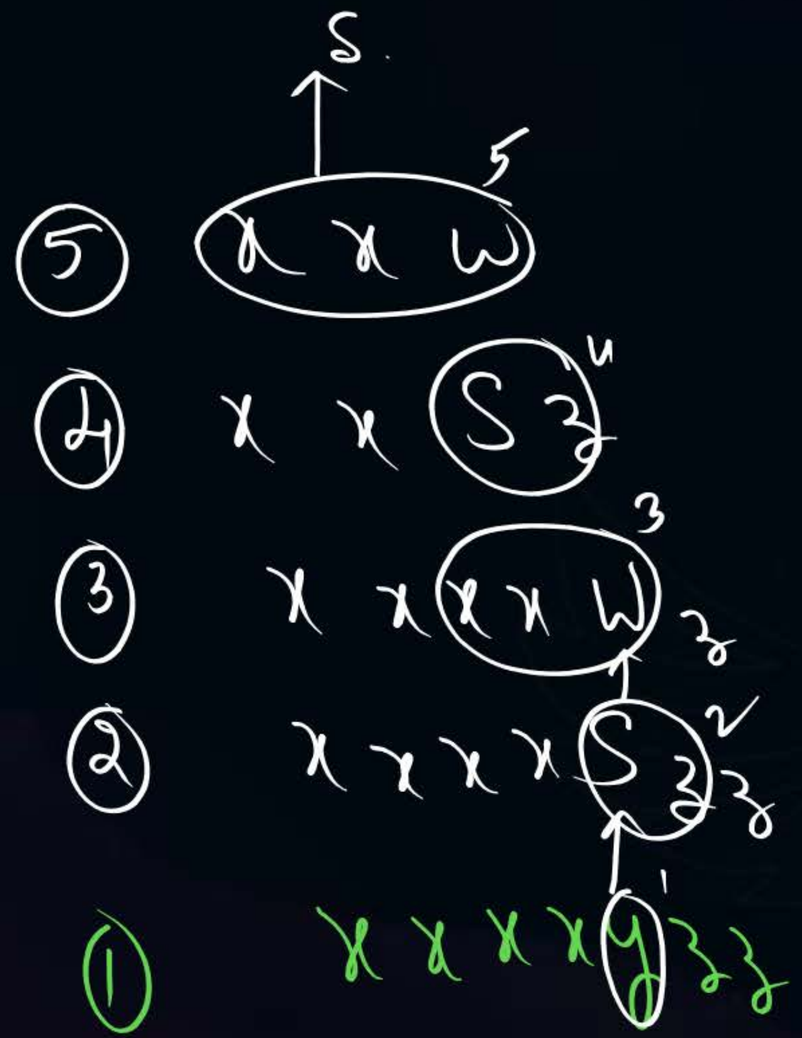
#Q. How many number of handles detected by bottom up parser for the following grammar and string.

$S \rightarrow xxW$

$S \rightarrow y$

$W \rightarrow Sz$

xxxxyzz

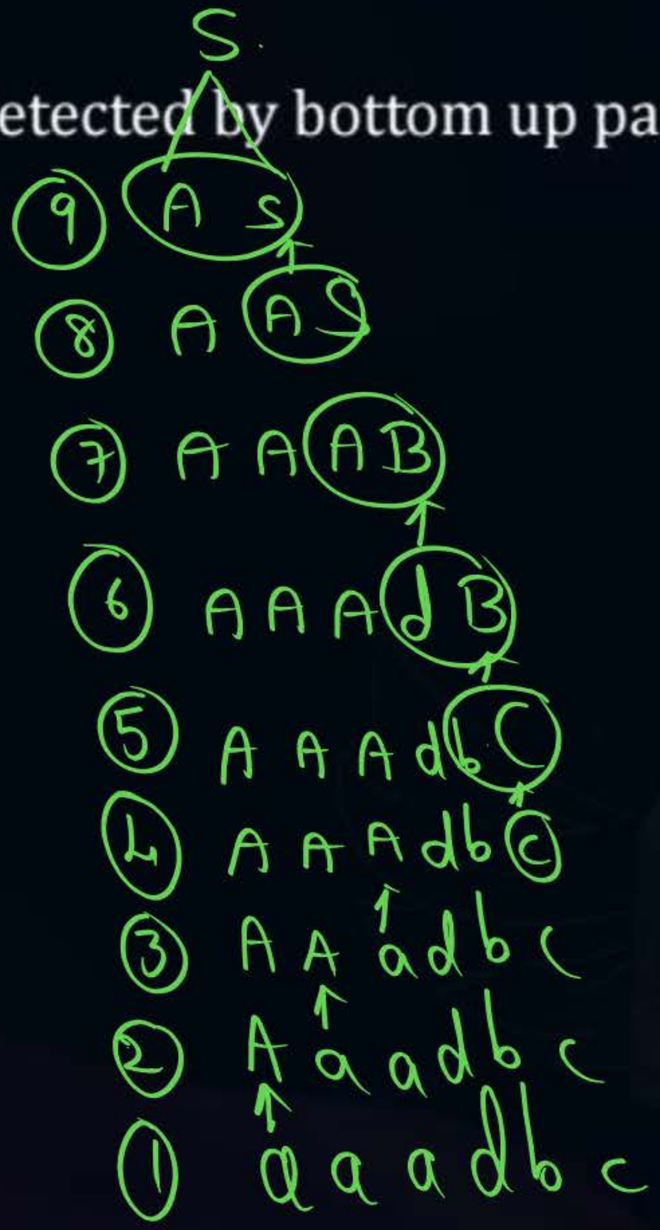


5 Handles

#Q. How many number of handles detected by bottom up parser for the following grammar and string.

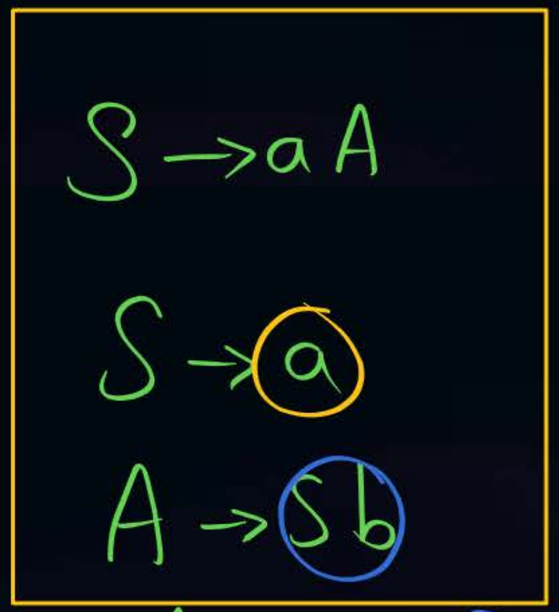
$S \rightarrow AS \mid AB$
$A \rightarrow a$
$B \rightarrow dB \mid bC$
$C \rightarrow c$

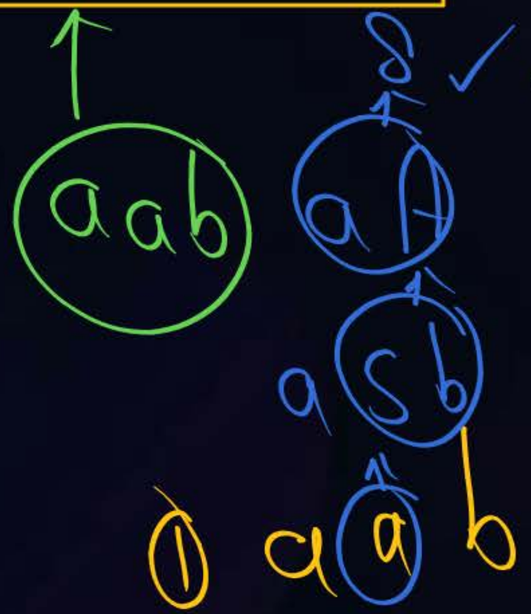
aaadbc



9 Handles

Which of the following Sequence of Handles detected by B·U·P





(a) a, aA, Sb

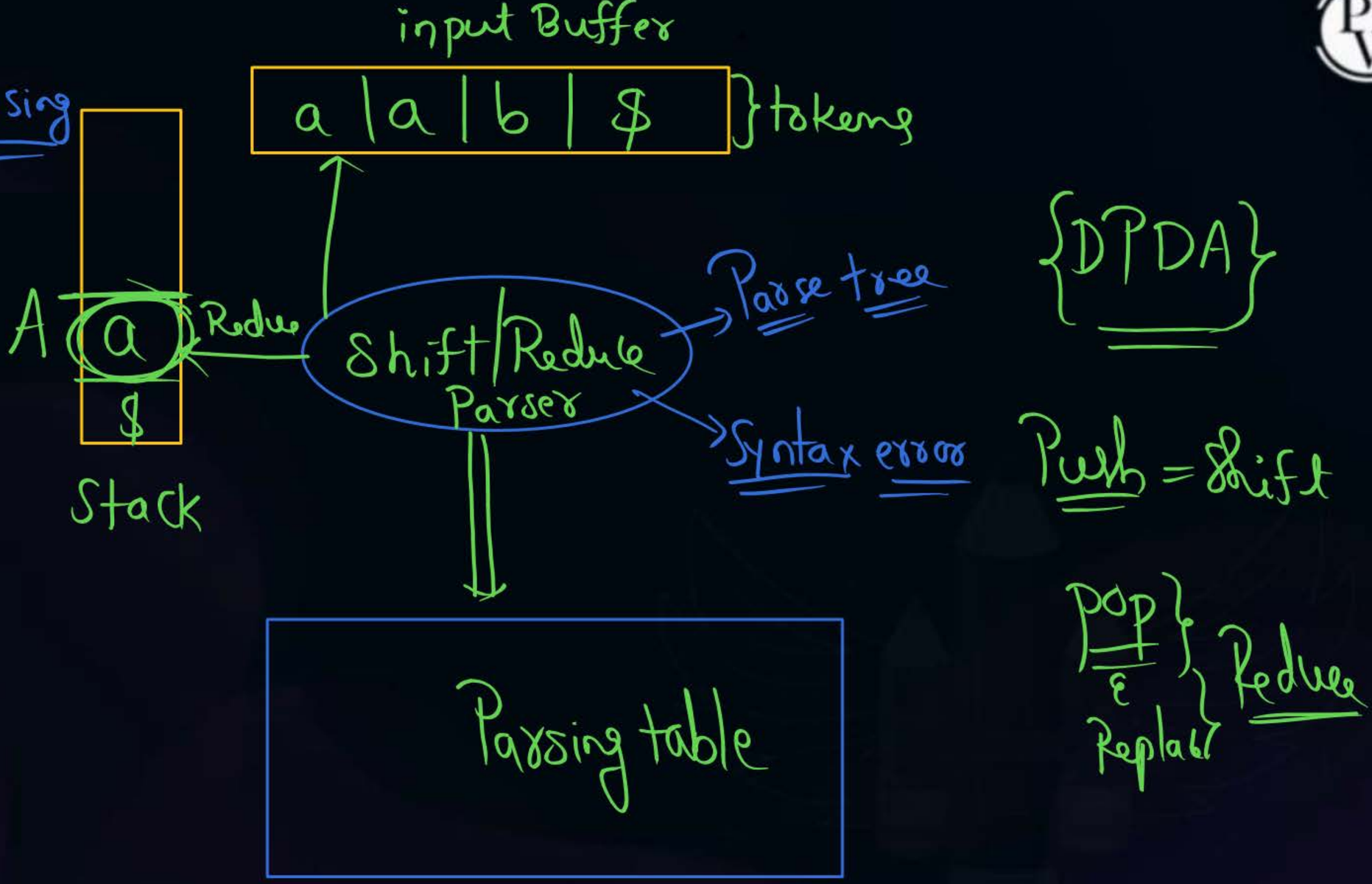
(b) a, Sb, aA ✓

(c) a, aA, a

(d) none

a, a, Sb ✓

Shift/Reduce Parsing



{DPDA}

Push = Shift

Pop
ε
Replace
} Reduce

Topic : Shift Reduce Parser in Compiler

- (1) Shift Reduce parser attempts for the construction of parse in a similar manner as done in bottom up parsing i.e. the parse tree is constructed from leaves(bottom) to the root(up).
- (2) A more general form of shift reduce parser is LR parser.
- This parser requires some data structures i.e.
- (3) A input buffer for storing the input string.
- (4) A stack for storing and accessing the production rules.

Topic : Shift Reduce Parser in Compiler

Basic Operations :-

- **Shift:** This involves moving of symbols from input buffer onto the stack.
- **Reduce:** If the handle appears on top of the stack then, its reduction by using appropriate production rule is done i.e. RHS of production rule is popped out of stack and LHS of production rule is pushed onto the stack.
- **Accept:** If only start symbol is present in the stack and the input buffer is empty then, the parsing action is called accept. When accept action is obtained, it is means successful parsing is done.
- **Error:** This is the situation in which the parser can neither perform shift action nor reduce action and not even accept action.

Topic : Shift Reduce Parser in Compiler

- Consider the grammar

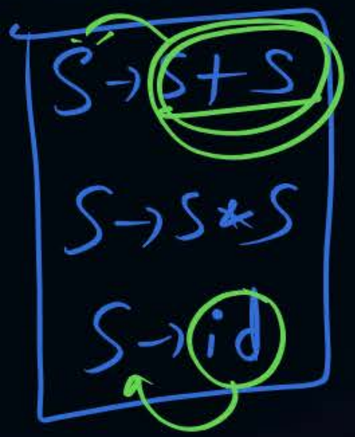
S –> S + S

S –> S * S

S –> id

- Perform Shift Reduce parsing for input string “id + id + id”.

Topic : Shift Reduce Parser in Compiler

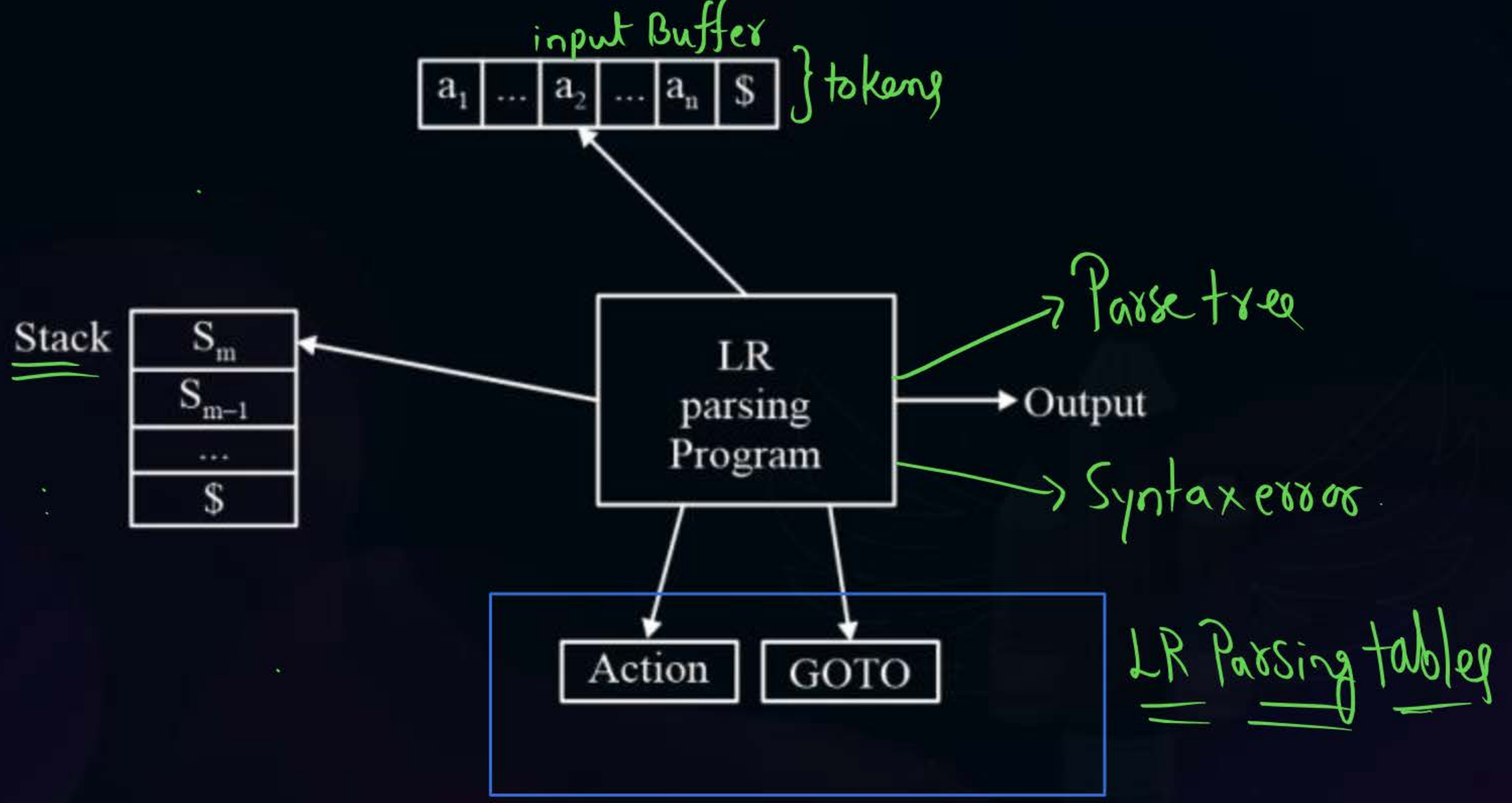


	Stack	Input Buffer	Parsing Action
1	$	id+id+id$	Shift
2	$id	+ id +id$	Reduce by S --> id
3	$S	+ id +id$	Shift
4	$S+	id+id$	Shift
5	$S+id	+id$	Reduce by S --> id
6	$S+S	+id$	Shift
7	$S+S+	id$	Shift
8	$S+S+id	$	Reduce by S --> id
9	$S+S+S	$	Reduce by S--> S + S
10	$S+S	$	Reduce by S--> S + S
11	$S	$	Accept

Shift
Reduce
Conflict

Topic : LR Parsing

Shift / Reduce Parser



#Q. How many shift action and reduce action are taken by LR parser to parse input ab by using following grammar and parsing table .

$S \rightarrow AB$

$A \rightarrow a$

$B \rightarrow b$

	a	b	$	S	A	B
I_0	S_3			1	2	
I_1			Accept			
I_2		S_5				4
I_3		r_2				
I_4			R_1			
I_5			r_3			

DFA

2 mins Summary

- **Topic** One
- **Topic** Two
- **Topic** Three
- **Topic** Four
- **Topic** Five

THANK - YOU